

Data Mining and Big Data Anlysis

数据挖掘及大数据分析

Chapter4：Data Preprocessing

数据预处理：00-概述

王东、张谦、丁玥

上海交通大学软件学院RFID与物联网实验室

2025年春季

概要

由于海量数据的来源是广泛的，数据类型也是多而繁杂的，因此，数据中会夹杂着不完整、重复以及错误的数据，如果直接使用这些原始数据的话，会严重影响数据决策的准确性和效率。因此，对原始数据进行有效的预处理是大数据分析和应用过程中的关键环节。

数据预处理的概念

数据挖掘领域

在数据挖掘领域中，数据清洗是数据进行预处理过程的第一个步骤。在数据预处理应用中，数据清洗的主要任务是提高数据的可用性，即去除噪声、无关数据以及空值等，并考虑数据的动态变化。在字符分类问题中，通过使用机器学习的技术进行数据清洗，即使用特定算法检测数据库，对缺失和错误的数据予以修改。

数据仓库领域

在数据仓库领域中，当多个数据库合并时或多个数据源进行集成时都需要进行数据清洗。例如，当同一个实体的记录在不同数据源中以不同的表示格式或错误表示的情况下，合并后的数据仓库中就会出现重复的记录，数据清洗的程序就需要识别出重复的记录并消除重复的记录，也就是所谓的数据合并或清除（即Merge/Purge）问题。在数据仓库环境中，数据清洗主要包括数据的清洗和结构的转换两个过程。

数据质量管理领域

在数据质量管理领域中，数据质量管理是一个学术界和商业界都感兴趣的领域。数据质量管理主要用于解决信息系统中的数据质量及集成问题。在该领域中，数据清洗从数据质量的角度出发，把数据清洗过程和数据生命周期集成在一起，对数据的正确性进行检查并提高数据质量。

Chapter 4: 数据预处理

- 概述

  —数据质量

  —数据预处理的主要任务

- 数据清洗（见数据预处理—数据清洗）
- 数据集成（见数据预处理—数据集成）
- 数据规约（见数据预处理—数据规约）
- 数据变换与数据离散化
- 小结

数据质量的概念

数据质量是指在业务环境下，数据符合数据消费者的使用目的，能满足业务场景具体需求的程度。但是，在不同的业务场景中，数据消费者对数据质量有着各自不同的观点



例如，对顾客地址列表，有些地址已经过时或不正确，只有80%的地址是正确的

对于销售经理，会认为数据是不正确的

对于市场分析人员考虑目标市场营销而言，这是一个大型顾客数据库，对该数据库准确性还算满意

数据质量是一个相对的概念



从适用性的角度来看，数据质量是一个相对的概念（与决策有关）。对于不同的决策者，大家对数据质量的高低要求也是不同的。对于一个无关的数据，即使质量很高，对决策也起不到任何作用。

例如，一个医院的病人基本信息通常包括姓名、年龄、血型、身高、地址等内容，如果想要研究某种疾病易发的年龄段，那么年龄信息的数据质量就非常的重要，而其它信息（即血型、身高、地址等信息）的数据质量相对来说作用不大。

数据质量: 为什么要对数据预处理?

- 衡量数据质量: 多种因素A multidimensional view
  - —准确性 Accuracy：数据是否正确、准确
  - —完整性 Completeness：未被记录，或者缺少属性
  - —一致性Consistency: 有的数据被修改了，有的没有
  - —时效性Timeliness: 数据是否及时更新?
  - —可信性Believability: 数据有多可信?
  - —可解释性Interpretability: 数据是否容易理解?

数据质量: 为什么要对数据预处理?

—准确性 **Accuracy**：数据是否正确、准确

- 收集数据的设备可能出故障
- 人或计算机的错误可能在数据输入时出现
- 当用户不希望提交个人信息时，可能故意向强制输入字段输入不正确的值（例如，为生日选择默认值“1 月1 日”），这称为被掩盖的缺失数据
- 错误也可能在数据传输中出现
- 不正确的数据也可能是由命名约定或所用的数据代码不一致，或输入字段（如日期）的格式不一致导致的
- 重复元组也需要数据清理

数据质量: 为什么要对数据预处理?

—完整性 Completeness：未被记录，或者缺少属性

- 有些感兴趣的属性，如销售事务数据中顾客的信息，并非总是可以得到的
- 有些数据没有包含在内，可能只是因为输入时认为是不重要的
- 相关数据没有记录可能是由于理解错误，或者因为设备故障
- 数据输入者不知道正确的值
- 数据值在稍后提供

- 缺失值并不意味数据有错误。例如，在申请信用卡时，可能要求申请人提供驾驶执照号，但没有驾驶执照的申请者可能自然地不填写该字段

数据质量: 为什么要对数据预处理?

—一致性Consistency: 有的数据被修改了，有的没有

- 与其他记录不一致的数据可能已经被删除
- 历史或修改的数据可能被忽略
- 缺失的数据，特别是某些属性上缺失值的元组，可能需要推导出来

数据质量: 为什么要对数据预处理?

—时效性Timeliness: 数据是否及时更新?

- 月底数据未能及时更新对数据质量具有负面影响
  - 一些销售代理未能在月末及时提交他们的销售记录，月底之后还有大量更正与调整。在下月的一段时间内，存放在数据库中的数据是不完整的
  - 然而，一旦所有的数据被接收之后，它就是正确的

数据质量: 为什么要对数据预处理?

—可信性Believability: 数据有多可信?

- 假设在某一时刻数据库有一些错误，之后被更正。然而，过去的错误已经给销售部门的用户造成了问题，因此他们不再相信该数据。

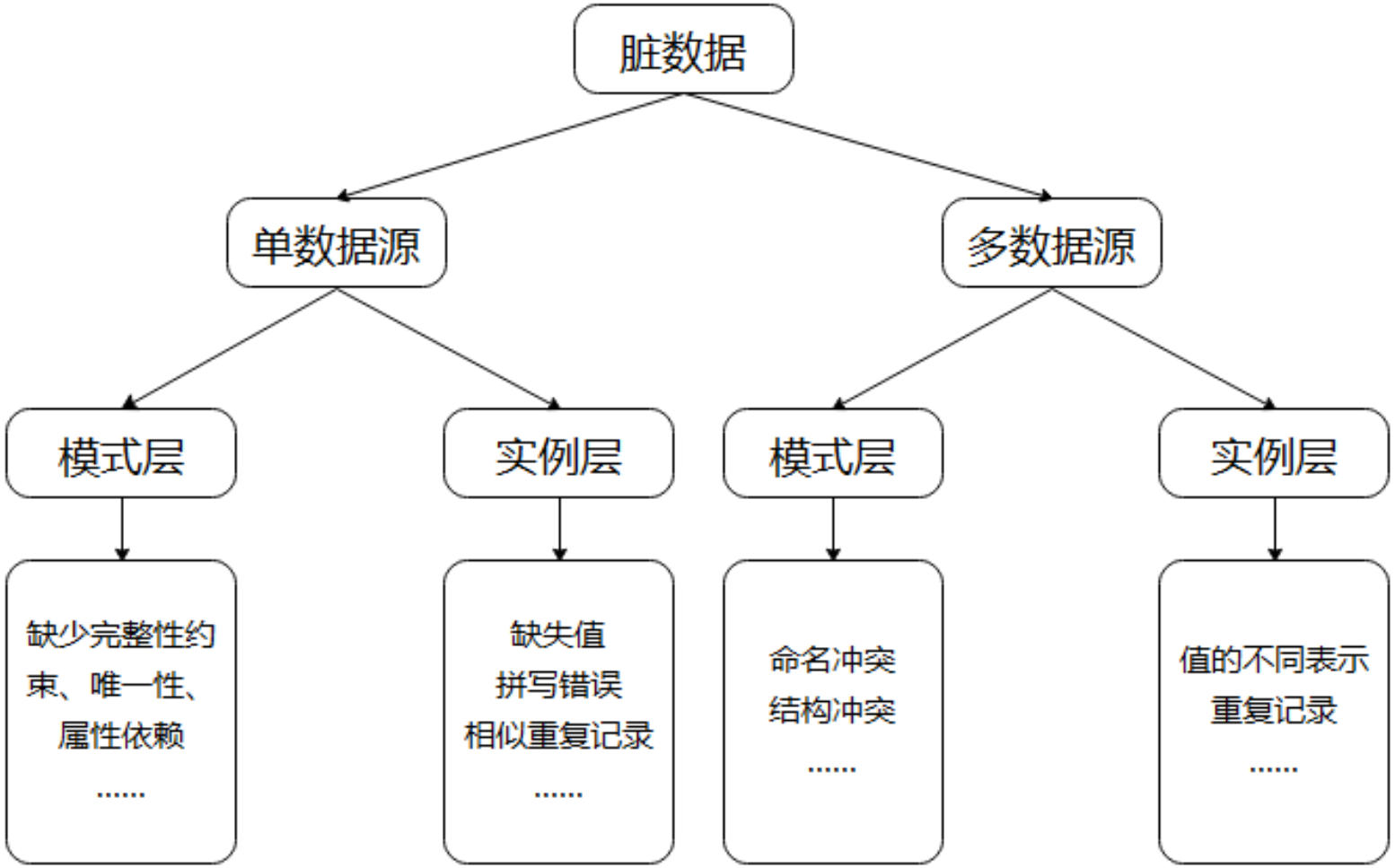
—可解释性Interpretability: 数据是否容易理解?

- 数据还使用了许多会计编码，销售部门并不知道如何解释它们

- **即使数据库现在是正确的、完整的、一致的、及时的，但是由于很差的可信性和可解释性，有些用户（销售部门）仍然可能把它看成低质量的数据**

数据质量的问题分类

基于数据源的脏数据分类



Chapter 4: 数据预处理

- 概述

  —数据质量

  —数据预处理的主要任务



- 数据清洗（见数据预处理—数据清洗）
- 数据集成（见数据预处理—数据集成）
- 数据规约（见数据预处理—数据规约）
- 数据变换与数据离散化
- 小结

数据预处理的主要任务

- 数据清洗Data cleaning
  - —填写缺失的值，光滑噪声数据noisy data，识别或删除离群点，解决不一致性
- 数据集成Data integration
  - —多个数据库、数据立方体或文件的集成
- 数据规约Data reduction
  - —维规约Dimensionality reduction
  - —数值规约Numerosity reduction
- 数据离散化和概念分层
  - —数据离散化
  - —概念分层Concept hierarchy generation
  - —数据规范化Normalization

数据预处理的主要任务

- 数据清洗Data cleaning
  - —填写缺失的值
  - —光滑噪声数据noisy data
  - —识别或删除离群点
  - —解决不一致性

数据预处理的主要任务

- 数据集成Data integration
  - —关于顾客标识的属性在一个数据库中可能是Custom_ID，而在另一个中为Cust_ID
  - —同一个人的名字可能在第一个数据库中登记为“ Bill”，在第二个数据库中登记为“ William，而在第三个数据库中登记为“ B ”
  - —有些属性可能是由其他属性导出的（例如，年收人〉，冗余数据可能降低数据挖掘过程的性能或使之陷入混乱

数据预处理的主要任务

- 数据规约Data reduction：得到数据集简化表示，小得多，但能够产生同样的（或几乎同样的）分析结果

—维规约Dimensionality reduction

- 使用数据编码方案，以便得到原始数据的简化或“ 压缩” 表示
  - 数据压缩技术（例如，小波变换和主成分分析）
  - 属性子集选择（去掉不相关的属性）
  - 属性构造（从原来的属性集导出更有用的小属性集）

—数值规约Numerosity reduction

- 使用最近邻分类或聚类这样的基于距离的挖掘算法进行分析
  - 顾客数据包含年龄和年薪属性，年薪属性的取值范围可能比年龄大得多，如果属性未规范化，则距离度量在年薪上所取的权重一般要超过距离度量在年龄上所取的权重
  - 如果待分析的数据已经规范化，即按比例映射到一个较小的区间（例如，[0 .0，1.0]），则这些方法将得到更好的结果

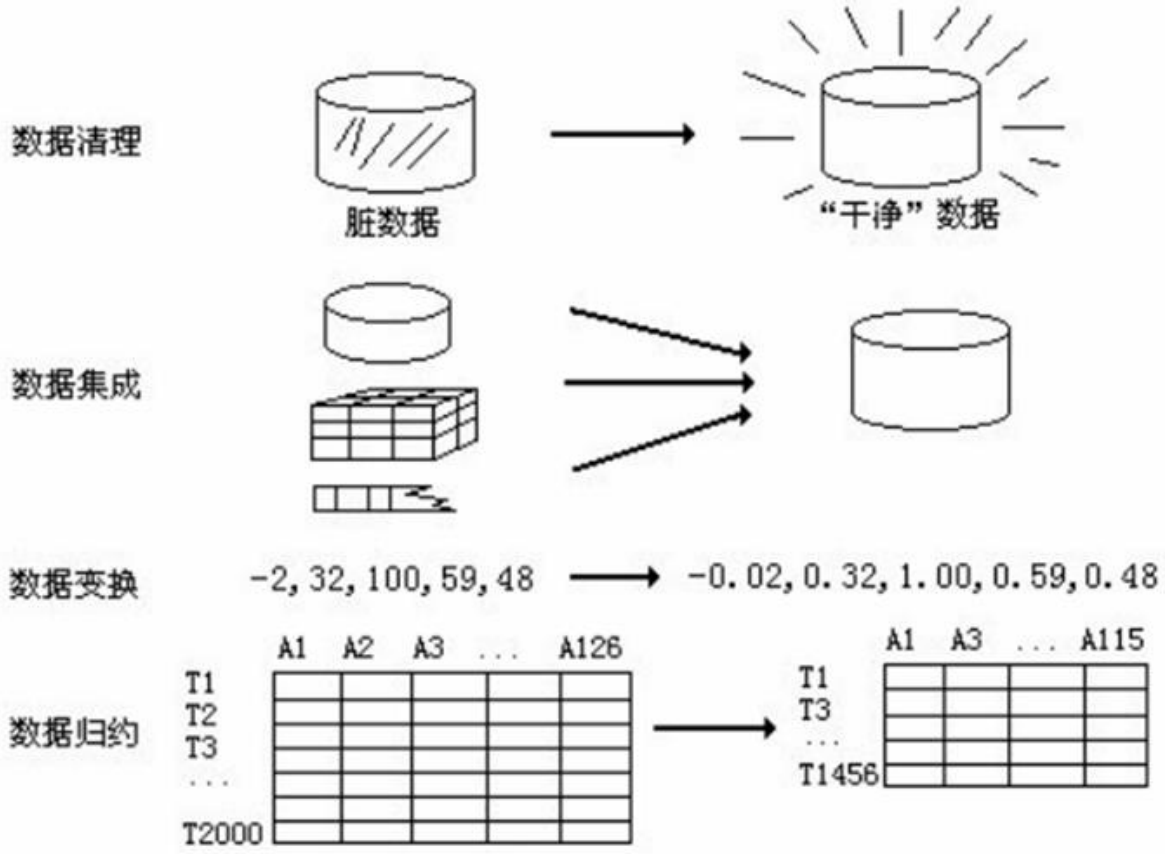
数据预处理的主要任务

- 数据离散化和概念分层

  —离散化和概念分层产生也可能是有用的，那里属性的原始值被区间或较高层的概念所取代。例如，年龄的原始值可以用较高层的概念（如青年、中年和老年）取代

数据预处理步骤

- 数据预处理分类不是互斥的。例如，冗余数据的删除既是一种数据清理形式，也是一种数据归约



Chapter 4: 数据预处理

- 数据预处理: 概述
- 数据清洗（见数据预处理—数据清洗）
- 数据集成（见数据预处理—数据集成）
- 数据规约（见数据预处理—数据规约）
- 数据变换与数据离散化
- 小结

Chapter 4: 数据预处理

- 数据预处理: 概述
- 数据清洗（见数据预处理—数据清洗）
- 数据集成（见数据预处理—数据集成）
- 数据规约（见数据预处理—数据规约）
- 数据变换与数据离散化
- 小结



数据变换Data Transformation

- 通过变换将整个数据集的给定属性的值映射到新的值，每个老的值都能通过新的值识别
- 方法
  - —光滑Smoothing: 去除数据中的噪声（数据清理的方法）
    - 分箱、回归、聚类
  - —属性构造Attribute/feature construction（数据规约方法）
    - 由给定的属性构造新的属性并添加到属性集中
  - —聚集Aggregation: 对数据进行汇总，构造数据立方体（数据规约方法）
  - —规范化Normalization: 把属性数据按比例缩放，使之落入特定大小区间
    - min-max normalization
    - z-score normalization
    - normalization by decimal scaling
  - —离散化Discretization: 组织更高层概念
- 数据预处理的主要任务之间存在许多重叠，上述策略中前三个在前面已经介绍

规范化Normalization

- 最小-最大规范化 Min-max normalization: to [new_min$_A$, new_max$_A$]

$$v' = \frac{v - min_A}{max_A - min_A}(new_max_A - new_min_A) + new_min_A$$

  - Ex. Let income range \$12,000 to \$98,000 normalized to [0.0, 1.0]. Then \$73,000 is mapped to $\frac{73{,}600 - 12{,}000}{98{,}000 - 12{,}000}(1.0 - 0) + 0 = 0.716$

- Z分数规范化Z-score normalization (μ: mean, σ: standard deviation):

$$v' = \frac{v - \mu_A}{\sigma_A}$$

  - Ex. Let μ = 54,000, σ = 16,000. Then $\frac{73{,}600 - 54{,}000}{16{,}000} = 1.225$

- 小数定标规范化Normalization by decimal scaling

$$v' = \frac{v}{10^j}$$ Where j is the smallest integer such that Max(|v'|) < 1

小数定标规范化（Decimal Scaling）是一种数据标准化方法，通过移动数据的小数点位置来进行标准化。小数点移动的位数取决于数据绝对值的最大值

离散化Discretization

- 属性有三种
  - 标称Nominal—values from an unordered set, e.g., color, profession
  - 序值Ordinal—values from an ordered set, e.g., military or academic rank
  - 数值Numeric—real numbers, e.g., integer or real numbers
- 离散化Discretization: 将连续的属性分割为有间隔的
  - 用间隔标签可以代替实际的数据值
  - 通过离散化减少数据大小
  - 监督的离散化Supervised vs. 非监督的离散化unsupervised
  - 自顶向下离散化Split (top-down) vs. 自底向上离散化merge (bottom-up)
  - 在一个属性值上可以递归地进行离散化
  - 为进一步的分析做准备（如分类）

数据离散方法

- 典型方法: 所有方法可以递归使用
  - —分箱Binning
    - Top-down split, unsupervised
  - —直方图Histogram analysis
    - Top-down split, unsupervised
  - —聚类分析Clustering analysis (unsupervised, top-down split or bottom-up merge)
  - —决策树分析Decision-tree analysis (supervised, top-down split)
  - —相关分析Correlation (e.g., χ^2) analysis (unsupervised, bottom-up merge)

简单离散化: 分箱

- 等宽Equal-width (distance) partitioning
  - Divides the range into *N* intervals of equal size: uniform grid
  - if *A* and *B* are the lowest and highest values of the attribute, the width of intervals will be: $W = (B - A)/N$.
  - 最直观，但易受离群点影响
- 等频Equal-depth (frequency) partitioning
  - Divides the range into *N* intervals, each containing approximately same number of samples
  - 良好的数据比例
  - 管理分类属性可能会非常棘手

分箱方法在数据光滑中的作用

- Sorted data for price (in dollars): 4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34

* Partition into equal-frequency (equi-depth) bins:
  - Bin 1: 4, 8, 9, 15
  - Bin 2: 21, 21, 24, 25
  - Bin 3: 26, 28, 29, 34
* Smoothing by bin means:
  - Bin 1: 9, 9, 9, 9
  - Bin 2: 23, 23, 23, 23
  - Bin 3: 29, 29, 29, 29
* Smoothing by bin boundaries:每个箱值被最接近的边界值替换
  - Bin 1: 4, 4, 4, 15
  - Bin 2: 21, 21, 25, 25
  - Bin 3: 26, 26, 26, 34



python 数据平滑的 Python |宁滨方法

数据平滑的 Python |宁滨方法

> 原文:https://www . geeksforgeeks . org/python-宁滨-数据平滑方法/

先决条件: ML |宁滨或离散化

宁滨方法用于平滑数据或处理有噪声的数据。在这种方法中，首先对数据进行排序，然后将排序后的值分配到多个桶或箱中。由于宁滨方法参考了值的邻域，所以它们执行局部平滑。

有三种方法来执行平滑–

> **通过面元手段平滑:**在通过面元手段平滑中，面元中的每个值都被面元的平均值所代替。 **按箱中值平滑:** 在此方法中，每个箱值都被其箱中值替换。 **按面元边界平滑:**在按面元边界平滑中，给定面元中的最小值和最大值被标识为面元边界。然后，每个箱值被最接近的边界值替换。

```
import numpy as np
import math
from sklearn.datasets import load_iris
from sklearn import datasets, linear_model, metrics

# load iris data set
dataset = load_iris()
a = dataset.data
b = np.zeros(150)

# take 1st column among 4 column of data set
for i in range (150):
     b[i]=a[i,1]

b=np.sort(b)  #sort the array

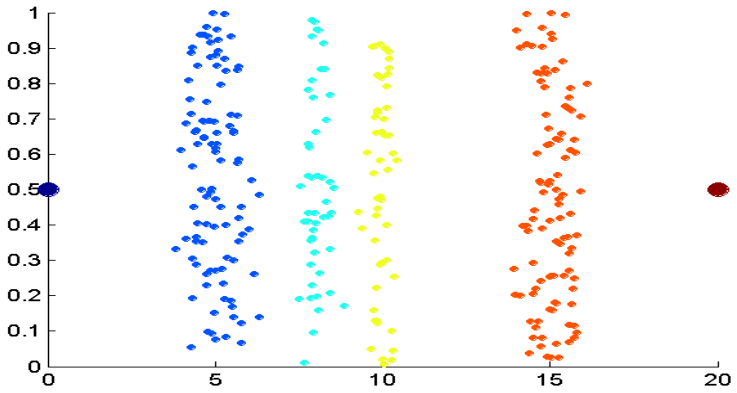
# create bins
bin1=np.zeros((30,5))
bin2=np.zeros((30,5))
bin3=np.zeros((30,5))

# Bin mean
for i in range (0,150,5):
     k=int(i/5)
     mean=(b[i] + b[i+1] + b[i+2] + b[i+3] + b[i+4])/5
     for j in range(5):
          bin1[k,j]=mean
print("Bin Mean: \n",bin1)

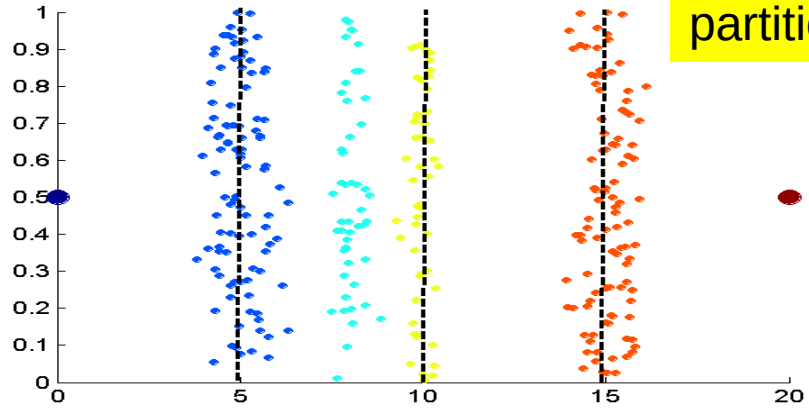
# Bin boundaries
for i in range (0,150,5):
     k=int(i/5)
     for j in range (5):
          if (b[i+j]-b[i]) < (b[i+4]-b[i+j]):
               bin2[k,j]=b[i]
          else:
               bin2[k,j]=b[i+4]
print("Bin Boundaries: \n",bin2)

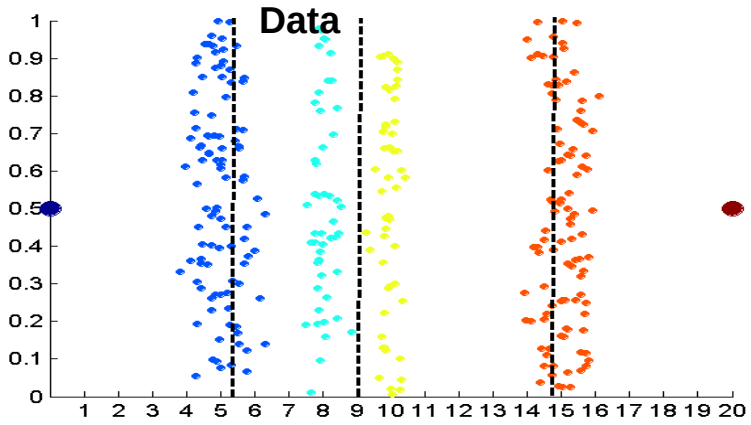
# Bin median
for i in range (0,150,5):
     k=int(i/5)
     for j in range (5):
          bin3[k,j]=b[i+2]
print("Bin Median: \n",bin3)
```

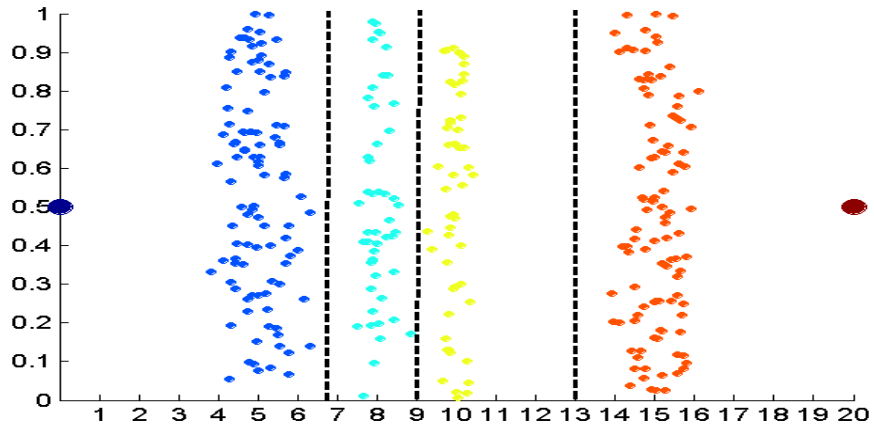
Discretization Without Using Class Labels (Binning vs. Clustering)



等宽Equal-width (distance) partitioning







等频Equal-depth

frequency) partitioning

Equal frequency (binning)

K-means clustering leads to better results

分类离散& 相关性分析

- 分类Classification (e.g.,决策树分析)
  - Supervised: Given class labels, e.g., cancerous vs. benign（像癌 不像癌）
  - Using 熵*entropy* to determine split point (discretization point)
  - Top-down, recursive split
  - Details to be covered in Chapter “Classification”
- 相关性分析Correlation analysis (e.g., Chi-merge: χ^2-based discretization)
  - Supervised: use class information
  - Bottom-up merge: find the best neighboring intervals (those having similar distributions of classes, i.e., low χ^2 values) to merge
  - Merge performed recursively, until a predefined stopping condition

概念分层产生Concept Hierarchy Generation

- 概念层次组织概念(i.e., attribute values)通常用在数据仓库中的每个维度相关联
- 概念层次结构便于在数据仓库中多粒度查看数据
- 概念层次的形成：递归通过收集和更高层次的概念（如青年，成年，或高级）取代低层次的概念（如年龄数值）减少数据
- 概念分层可以由该领域的专家和/或数据仓库设计人员显式地指定
- 概念分层可以为数值数据和标称数据自动形成，对于数字数据，显示使用离散化方法

标称数据的概念分层产生

- 由用户或专家在模式级显式地说明属性的部分序
  - —***street < city < state < country***
- 通过显式数据分组说明分层架构的一部分
  - —**{Urbana, Champaign, Chicago} < Illinois**
- 说明属性集但不说明它们的偏序
  - —**E.g., only *street* < *city*, not others**
- 根据每个属性的不同值的个数产生概念分层
  - —**E.g., for a set of attributes: {*street, city, state, country*}**

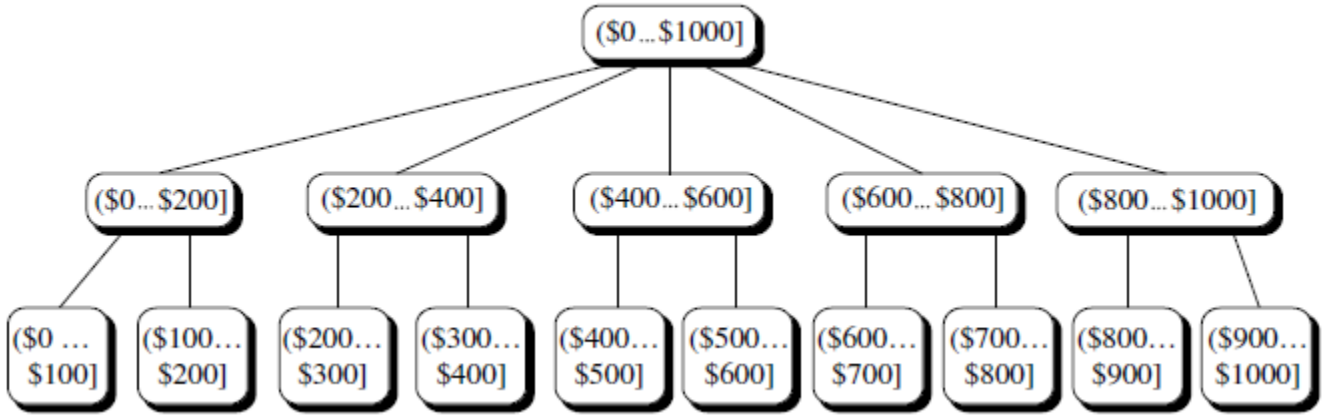
自动生成概念分层

- 有时候，概念分层可以分析数据集中每个属性不同值的个数自动生成
  - —个数最多的属性被放在概念分层的最底层
  - —**Exceptions, e.g., weekday, month, quarter, year**



概念分层

- 可以将给定维或属性的值离散化或分组来定义概念分层，产生集合分组分层
- 可以在值的组之间定义全序或偏序
- 以下是关于维price的集合分组概念分层



- 对于一个给定的属性或维，按照不同的用户观点，可能有多个概念分层
  - 用inexpensive，moderately_priced和expensive来组织price

Chapter 4: 数据预处理

- 数据预处理: 概述
- 数据清洗（见数据预处理—数据清洗）
- 数据集成（见数据预处理—数据集成）
- 数据规约（见数据预处理—数据规约）
- 数据变换与数据离散化
- 小结



Summary

- Data quality: accuracy, completeness, consistency, timeliness, believability, interpretability
- Data cleaning: e.g. missing/noisy values, outliers
- Data integration from multiple sources:
  - —Entity identification problem; Remove redundancies; Detect inconsistencies
- Data reduction
  - —Dimensionality reduction; Numerosity reduction; Data compression
- Data transformation and data discretization
  - —Normalization; Concept hierarchy generation